

To: Eclipse Catering

From: G-Solutions² (godlinessdongorere@gmail.com)

Date: September 22, 2026

Report Ref: CATERFLOW-RETENTION-2026-09-22-R2

Supersedes: CATERFLOW-RETENTION-2026-09-22

Sanity Document Retention — Corrected Findings & Fix

CaterFlow production dataset (Sanity project v3sfsml, dataset "production")

Correction to the previous report

The prior report (Ref CATERFLOW-RETENTION-2026-09-22) concluded the retention/deletion pipeline was working as designed and that document volume was "comfortably within limit." Both conclusions were wrong. The production app has since become blocked by Sanity's document quota, which prompted this follow-up. A deeper investigation found two real defects in the archive-and-delete pipeline — not a self-resolving reference-ordering delay. Both have been identified, fixed in code, and verified with tests and a production build, but **not yet deployed**.

This update replaces the root-cause analysis in the previous report. The earlier audit measured the deletion pipeline's real output (roughly 558 documents clearing its eligibility gate) and, because that number looked small and reference-ordering errors were being logged for the rest, concluded the low count was expected, self-resolving behavior. It was not: the eligibility gate itself was checking the wrong date field, and was suppressing deletion of the vast majority of documents that should already have been removed under the stated 60-day retention policy.

TOTAL DOCUMENTS (LIVE)

31,059

DELETABLE BEFORE FIX

586

DELETABLE ONCE FIXED

11,943

Summary

Two defects in the archive-and-delete pipeline were found and fixed this session:

- **Wrong eligibility gate (the actual reason nothing was being deleted).** The deletion step compared `_archivedAt` — the timestamp a document was copied into the Mongo archive — against the 60-day retention cutoff, instead of the document's own business date (`dispatchDate` , `orderDate` , etc.), which the code's own design comments say it should use. Since a large share of documents were only (re-)copied into Mongo very recently — notably around the September 21 archive-database migration — almost everything looked "too recently archived" to delete, regardless of how old the underlying record actually was.
- **Lost resume progress on the archive step.** The multi-step archive process (8 document types, several thousand documents) only remembered where it left off if the exact same internal run ID was reused across every retry. Any time a new run ID was issued — including automatically, whenever a run went 5 minutes without an update — all progress was discarded and the entire process restarted from the first step. This wasted significant processing time re-scanning documents that were already handled, and slowed how quickly documents became archived (a prerequisite for deletion) in the first place.

Both are now fixed in `src/lib/archiveService.ts` , covered by updated/new automated tests (22 passing), and confirmed to build cleanly. The fix has not been deployed or run against production yet — that step needs a conscious go-ahead, since the very next cleanup run after deploying will attempt to permanently delete a very large batch of documents (see Recommended Next Steps).

What the Previous Report Got Wrong

The previous report's own numbers were correct as measurements — it just misattributed their cause:

- Its "558 eligible for deletion now" is explained almost exactly by this session's direct measurement of the same (buggy) gate: **586 documents** currently pass the `_archivedAt`-based check (the small difference is normal drift — new documents crossing the archive-copy cutoff — between the two measurements).
- Its "11,946 overdue by business date" figure was itself accurate and, read correctly, was the real number that should have been deletable. This session's equivalent measurement against the corrected, business-date-based gate returns **11,943** — consistent with that figure, confirming the previous audit's raw data was sound even though its conclusion about what was blocking deletion was not.
- The "reference-order blocking" explanation (PurchaseOrders waiting on their GoodsReceipt) is real and still correct as far as it goes — it is just not the reason deletion volume was low. It only affects the ordering of an already-small set; it does not explain why 11,000+ other overdue documents were never attempted.

Root Cause Detail

1. Deletion gate used the wrong clock

Every archive step already copies a document's own business date into its Mongo record (e.g. `dispatchDate` for DispatchLogs, `orderDate` for PurchaseOrders). The deletion step ignored these and instead checked `_archivedAt` — when the Mongo copy itself was made. A document is normally archived close to when it's created, so in steady state these two clocks track each other closely and the bug stays invisible. They diverge sharply after any event that causes a large batch of documents to be (re-)archived long after their original date — which is exactly what happened around the September 21 Mongo archive-database migration. The result: genuinely old records were locked out of deletion for another full retention period, measured from the wrong starting point.

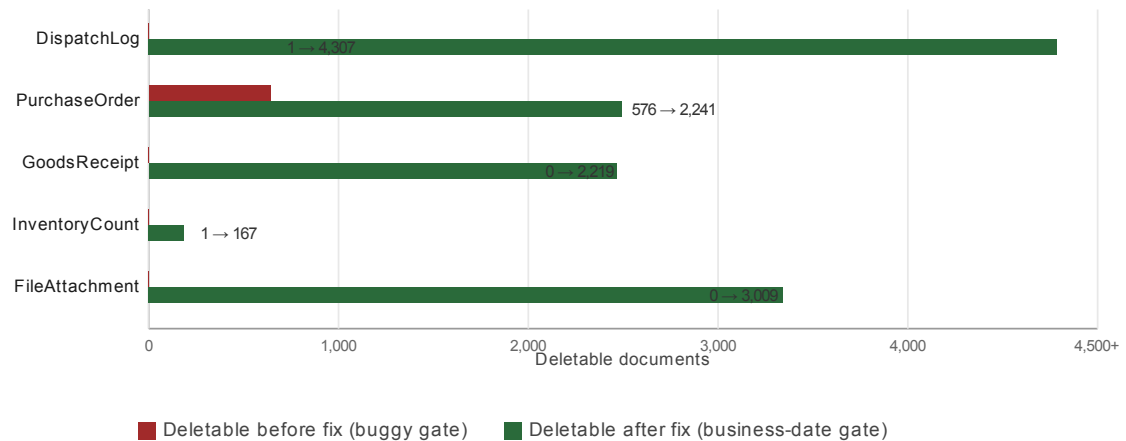
2. Archive progress reset on every new run ID

The archive step processes 8 document types in sequence and checkpoints its position so a run interrupted by Vercel's execution time limit can resume where it left off. That resume logic only worked if the next attempt reused the identical internal run ID. In practice, a fresh run ID could be issued by more than one path — including a background process that automatically marks a run "failed" after 5 minutes without an update, purely as a liveness check. Whenever that happened, the next attempt had no matching history to resume from and restarted the entire 8-step sequence from the beginning, re-checking documents it had already processed. This did not cause incorrect deletions or data loss, but it meant the archive step took much longer than necessary to get every eligible document copied into Mongo in the first place.

Impact by Collection

Deletable document counts, before and after the fix, measured directly against production Sanity and Mongo data on September 22, 2026, using the code's 60-day retention default (cutoff: 2026-07-24).

Deletable documents: before fix vs. after fix



Collection	Total Archived	Deletable Before Fix	Deletable After Fix
DispatchLog	7,103	1	4,307
PurchaseOrder	3,255	576	2,241
GoodsReceipt	3,104	0	2,219
InternalTransfer	0	0	0
StockAdjustment	8	8	0
InventoryCount	256	1	167
FileAttachment	9,908	0	3,009
StockSnapshot	0	0	0
TOTAL	23,634	586	11,943

"Deletable" requires the archived Mongo copy to exist, its business date to be past the 60-day cutoff (2026-07-24), and — for workflow-tracked types — its status to be a finished state (completed/cancelled/adjusted); draft and in-progress records are correctly excluded either way. Note StockAdjustment drops from 8 to 0 under the fix: those 8 records had old `_archivedAt` timestamps but are not actually old enough by their own `adjustmentDate` — the fix is more conservative here, correctly preventing premature deletion, which is the same underlying correction working in the opposite direction.

Fix Status

- **Implemented:** `cleanupCollectionBatched()` now filters on each collection's own business-date field via a new `CLEANUP_DATE_FIELD` mapping, instead of `_archivedAt`.
- **Implemented:** the archive step's resume logic now recovers the last checkpoint from any genuinely incomplete prior attempt, not only one sharing the exact same run ID — while still correctly refusing to re-run a step that a completed run already finished.
- **Verified:** full TypeScript type-check clean; production build (`next build`) succeeds; existing cleanup-query test updated to assert the correct field, plus a new regression test covering all 8 collections; 22/22 tests passing.
- **Not yet done:** deploying the fix, and running an actual archive + delete pass against production.

Recommended Next Steps

- **Take a fresh backup before deploying.** The very next cleanup run after this fix ships will attempt to permanently delete on the order of 11,900 documents in one pass, rather than the handful seen previously — a backup immediately beforehand is

cheap insurance.

- **Deploy, then trigger one archive run and one delete run, and watch the result before assuming it's resolved.** Confirm the actual deleted-document count matches expectations and that no unexpected errors appear, rather than treating "it ran without crashing" as sufficient.
- **Confirm which `ARCHIVE_DAYS_THRESHOLD` value is actually set in the production environment.** The local development environment used for this analysis has it set to 90 days; the code's built-in default (and what the previous report assumed) is 60. This session's headline numbers use the 60-day default for direct comparability with the previous report — confirm which one is actually live before treating either figure as final.
- **Re-check the Sanity account's actual document quota and plan tier directly on the billing page.** The previous report's assumption of a 50,000-document Growth-tier limit is contradicted by the app now being blocked at roughly 31,000 documents — either the tier assumption was wrong, or a lower effective limit applies. This should be confirmed rather than assumed.
- **After the first full delete pass, re-run this same before/after comparison** to confirm the true remaining backlog has actually shrunk, rather than relying on the admin dashboard's per-pass "skipped" counts, which mostly reflect re-verification scans rather than net progress.

Why this matters beyond the current quota block

Because the deletion gate has effectively never worked as intended, the dataset has been growing without the retention policy actually enforcing itself since it was introduced. Fixing the gate is necessary but not sufficient on its own — once the current backlog is cleared, it's worth confirming the fixed pipeline keeps pace with new documents aging past the retention window on an ongoing basis, so this does not recur.